

# Conceptual Design Virtual Worker

**System Name: Virtual Worker Platform (VWP)**

**Technologies: n8n + Hugging Face**

---

## 1. Document Control

### 1.1 Document Information

- **Document Title:** Conceptual Design Document
- **System Name:** Virtual Worker Platform (VWP)
- **Version:** 1.0
- **Author:** To be assigned
- **Reviewers:** Architecture Review Board, Security & Compliance Lead, Automation Lead
- **Approval Authority:** Program Sponsor / CTO
- **Date:** 16/02/2026
- **Classification:** Internal Engineering Document

### 1.2 Revision History

| Version | Date       | Author         | Description of Changes             |
|---------|------------|----------------|------------------------------------|
| 1.0     | 16/02/2026 | To be assigned | Initial Conceptual Design Baseline |

### 1.3 Distribution List

- Engineering Team
  - Architecture Review Board
  - Governance & Compliance
  - Program Sponsor
- 

## 2. Introduction

### 2.1 Purpose

This Conceptual Design Document defines the mission, operating concept, and high-level architecture for the Virtual Worker Platform (VWP). The document provides a blueprint for how the platform will safely

convert natural-language tasks into governed, auditable workflow executions using Hugging Face for AI interpretation and n8n for workflow automation.

## 2.2 Scope

### Included

- Natural language task intake (UI and API channels)
- AI interpretation (classification, entity extraction, summarization where required)
- Template-driven workflow mapping
- Workflow execution orchestration through n8n
- Approval gating and policy enforcement
- Audit logging and traceability
- Multi-tenant workspace isolation and RBAC
- Core integrations (email, calendar, CRM, HTTP APIs, databases)

### Excluded

- Unrestricted autonomous agent execution without controls
- Desktop RPA / screen automation (unless explicitly added later)
- Financial transaction execution without approvals
- Open-ended web browsing agents operating outside defined templates

## 2.3 Definitions & Acronyms

- **VWP** – Virtual Worker Platform
- **n8n** – Workflow automation/orchestration platform
- **HF** – Hugging Face
- **RBAC** – Role-Based Access Control
- **CONOPS** – Concept of Operations
- **NFR** – Non-Functional Requirement
- **ICD** – Interface Control Document
- **HITL** – Human-in-the-Loop (approvals)

## 2.4 References

- Virtual Worker BRS (internal)
  - n8n product documentation (execution, credentials, queue mode)
  - Hugging Face inference approach (hosted or self-hosted)
-

## 3. System Overview

### 3.1 System Mission

Provide a governed, enterprise-ready digital workforce layer that accepts natural language tasks and completes them through controlled, auditable workflow execution across business systems.

### 3.2 System Objectives

1. Reduce manual effort for repetitive digital tasks by at least 50%.
2. Execute tasks reliably using template-constrained workflows with deterministic outcomes.
3. Ensure safety and compliance using policy enforcement, approvals, and full audit trails.
4. Enable rapid automation development by standardizing templates and n8n workflow conventions.
5. Scale across departments and organizations using multi-tenant controls.

### 3.3 High-Level Description

VWP is a multi-tenant task execution platform. Users submit tasks via UI/API. The platform interprets tasks using Hugging Face models to select a pre-approved template and extract structured inputs. The backend enforces policy (RBAC, approval requirements, allowed actions), then triggers n8n workflows to execute integrations. The platform captures outputs, artifacts, and audit evidence for each run.

### 3.4 Design Drivers

- **Governance first:** approvals, least privilege, traceability
  - **Template constrained execution:** avoid unbounded AI action
  - **Separation of concerns:** AI proposes, platform governs, n8n executes
  - **Enterprise readiness:** multi-tenancy, RBAC, auditability
  - **Scalability:** queue-based execution and horizontal workers
- 

## 4. Concept of Operations (CONOPS)

### 4.1 Operational Environment

- Deployed as containerized services (cloud or on-prem)
- Used by individuals and organizations (workspaces)
- Integrates with enterprise systems through APIs (Email, Calendar, CRM, DB)
- Operates under defined governance rules (approval gates, audit trails)

### 4.2 Stakeholders

- Program Sponsor / Executive Owner
- Product Owner

- Engineering Team (Backend, Automation, AI)
- Security & Compliance
- Business Admins (workspace admins)
- End Users (task submitters)
- Approvers (managers, compliance reviewers)

## 4.3 Operational Scenarios

### Normal Operation (Happy Path)

1. User submits a task (text + optional attachments) via UI/API.
2. System classifies the task and selects the most suitable approved template.
3. System extracts required inputs into validated structured JSON.
4. System produces a plan preview (steps + approvals required, if any).
5. Policy engine evaluates risk and determines whether approval is needed.
6. If no approval: workflow is executed in n8n; else task pauses for approval.
7. System records outputs and artifacts; user receives completion notification.

### Failure Scenario

- **Trigger:** Workflow connector fails (e.g., email provider down) or validation fails.
- **Expected Response:**
  - System marks run as failed with error code and evidence
  - Retries are attempted based on policy (idempotent actions only)
  - User and/or admin is notified
  - Run is available for re-execution after issue remediation

### Edge Cases

- Low confidence classification: user must confirm template or provide missing fields.
- Duplicate submission: idempotency prevents double execution.
- Permission mismatch: RBAC denies execution and logs event.

## 4.4 Operational Modes

- **Intake Mode:** receive tasks through channels
  - **Interpretation Mode:** classify and extract inputs
  - **Planning Mode:** build plan preview + risk flags
  - **Governance Review Mode:** approvals and policy checks
  - **Execution Mode:** run n8n workflow
  - **Publishing Mode:** deliver outputs/artifacts
  - **Maintenance Mode:** template updates, credential rotation, monitoring
-

## 5. System Context

### 5.1 Context Description

VWP sits between end users and enterprise systems as a governed automation layer.

#### Upstream Systems (Task Sources)

- Web UI
- API clients
- Email-to-task (optional)
- Slack/Teams (optional)

#### Downstream Consumers

- End Users (results, artifacts)
- Business teams (reports, CRM updates)
- Admins (dashboards, audit exports)

#### Supporting Systems

- Identity Provider (SSO/OAuth, optional in MVP)
- PostgreSQL database
- Object storage (S3 compatible)
- Secrets vault / n8n credential store
- Logging/Monitoring stack

### 5.2 External Interfaces

- RESTful APIs (platform API)
  - n8n webhook/execution APIs
  - Data connectors (email/calendar/CRM/DB)
  - Identity provider integration (OAuth/SSO)
  - Secure storage interfaces (S3, KMS)
- 

## 6. Functional Architecture

### 6.1 Level 0 – System

**Virtual Worker Platform:** Interpret → Govern → Execute → Prove.

### 6.2 Level 1 – Major Subsystems

- Task Intake & Management

- **Template Registry & Versioning**
- **AI Interpretation Service (HF)**
- **Policy & Governance Engine**
- **Approval & Workflow Pause/Resume**
- **Execution Orchestrator (n8n bridge)**
- **Audit & Traceability Engine**
- **Notification Service**
- **Reporting & Analytics**

## **6.3 Level 2 – Module Decomposition**

### **Task Intake & Management**

- Channel adapters (UI/API/Slack/Email)
- Task state machine
- Attachment handler

### **Template Registry & Versioning**

- Template CRUD
- Input schema store (JSON schema)
- Version control and rollback

### **AI Interpretation Service (HF)**

- Classifier (template routing)
- Entity extractor (structured inputs)
- Summarizer (optional per template)
- Output normalizer (JSON validation)

### **Policy & Governance Engine**

- Approval rules evaluator
- Risk scoring / flags
- RBAC controller
- Rate limiting & thresholds

### **Approval & Workflow Pause/Resume**

- Approval request generator
- Approver UI/API
- Resume/abort controller (signals n8n)

### **Execution Orchestrator (n8n bridge)**

- Workflow launcher
- Execution tracker
- Idempotency controller
- Output/artifact collector

## Audit & Traceability Engine

- Immutable event logger
- Evidence pointer generator (links to payloads/artifacts)
- Run-to-template traceability mapping

## Notification Service

- Email notifications
- Slack/Teams notifications

## Reporting & Analytics

- Operational dashboards
- Governance dashboards
- AI quality dashboards

## 6.4 Functional Flow

Task Request → Classification → Template Selection → Entity Extraction → Schema Validation → Policy Evaluation → (Approval if required) → n8n Execution → Output Capture → Audit Logging → Notification / Publication

---

# 7. Logical Architecture

## 7.1 Architectural Layers

- **Experience Layer:** UI and channels
- **Control Layer:** backend APIs + policy
- **Intelligence Layer:** HF inference services
- **Execution Layer:** n8n workflows and connectors
- **Evidence Layer:** audit logs and run artifacts
- **Data Layer:** Postgres + object storage

## 7.2 Major Logical Components

- Task API + State Machine
- Template Service + Schema Registry
- AI Gateway (HF)
- Policy Engine (RBAC + approvals + limits)
- n8n Bridge (execution + callbacks)
- Audit/Event Store
- Notification Engine
- Analytics Engine

## 7.3 Data Flow Description

1. User submits task → stored in Task store
  2. Task text/attachments → AI Gateway for classification/extraction
  3. Extracted inputs → validated against template schema
  4. Policy Engine determines approvals and execution eligibility
  5. n8n executes using workspace-scoped credentials
  6. Outputs/artifacts returned → stored and linked to task/run
  7. Audit events emitted at each step
- 

## 8. Data Architecture

### 8.1 Core Data Objects

- Workspace
- User
- Role/Permission
- Task
- Template
- TemplateVersion
- Run
- Approval
- AuditEvent
- Attachment
- Artifact
- IntegrationCredentialRef (reference only; secrets remain in vault)

### 8.2 Common Data Model (CDM)

The platform uses a **Task Execution CDM** to standardize intent and execution:

- **TaskIntent:** {category, template\_id, confidence, rationale}
- **TaskInputs:** validated JSON per template schema
- **ExecutionPlan:** ordered steps + approval checkpoints
- **ExecutionOutcome:** status, outputs, artifacts, evidence pointers

**Naming conventions:** snake\_case keys; stable IDs; versioned schemas. **Standard types:** string, number, boolean, arrays, timestamps (ISO 8601). **Required attributes:** workspace\_id, created\_by, timestamps, status.



## 8.3 Lineage & Traceability Model

Each run must maintain traceability to:

- workspace\_id and actor
- selected template\_id + template\_version
- AI outputs (classification/extraction) with hashes
- n8n workflow\_id + execution\_id
- approval decisions (who/when/why)
- artifacts and outputs (links + hashes)

## 8.4 Versioning Strategy

- Template versioning (immutable versions; active pointer)
  - Workflow version alignment (template\_version maps to workflow\_version)
  - Immutable audit history
  - Change log retention and rollback capability
- 

# 9. Interface Architecture

## 9.1 Internal Interfaces

- Task Service → AI Gateway
- AI Gateway → Template Service
- Template Service → Policy Engine
- Policy Engine → Approval Service
- Policy Engine → n8n Bridge
- n8n Bridge → Audit Engine
- n8n Bridge → Notification Service

## 9.2 External Interfaces

- Versioned REST APIs for tasks/templates/runs/approvals/audit
- n8n webhooks for execution start/stop/callback
- Source system connectors (email/calendar/CRM/DB)
- Identity provider integration (OAuth/SSO)
- Object storage integration (S3)

## 9.3 Interface Control Principles

- Explicit schema contracts (JSON schema)
- Version-controlled APIs
- Backward compatibility enforcement
- Strict validation at boundaries

- Idempotency keys for execution triggers
  - Redaction rules for sensitive fields
- 

## **10. Non-Functional Architecture**

### **10.1 Security**

- RBAC enforcement
- Workspace isolation on every object
- Encryption in transit (TLS)
- Encryption at rest (DB + object storage)
- Immutable audit logs
- Secrets stored only in vault / n8n credentials
- Approval gates for sensitive actions

### **10.2 Performance**

- Horizontal scalability for execution workers
- Stateless backend where possible
- AI interpretation target: < 5 seconds (typical cases)
- Support for batch execution (later phase)

### **10.3 Reliability**

- Idempotent execution operations
- Retry and rollback mechanisms
- Timeouts on long-running workflows
- Backup and recovery strategy (DB + object storage)

### **10.4 Deployment Model**

- Cloud-native containerized services
- CI/CD controlled releases
- Environment segregation (Dev/UAT/Prod)
- Secrets management via vault/KMS

### **10.5 Compliance & Governance**

- PII tagging (where applicable)
  - Retention policy enforcement
  - Access logging and audit review
  - Approval evidence retained with run record
-

# 11. Constraints & Assumptions

## 11.1 Assumptions

- Required integration APIs are available and authorized.
- n8n supports target connectors (or HTTP integration is acceptable).
- HF inference is available (hosted initially; self-host optional later).
- Users have network access to the platform.

## 11.2 Constraints

- Budget constraints may influence hosted vs self-hosted AI.
  - Regulatory requirements may require on-prem or data residency.
  - MVP timeline limits number of integrations/templates.
- 

# 12. Risk Considerations

## 12.1 Technical Risks

- AI misclassification leading to wrong template selection.
- Workflow idempotency gaps leading to duplicate actions.
- Integration API rate limits and intermittent failures.

## 12.2 Operational Risks

- Slow approvals impacting throughput.
- Inadequate RBAC leading to over-permissioning.
- Template sprawl without governance.

## 12.3 Mitigation Strategies

- Confidence thresholds + user confirmation on low confidence.
  - Strict schema validation and policy enforcement.
  - Mandatory approvals for high-risk actions.
  - Idempotency keys and step-level execution markers.
  - Monitoring/alerting and connector fallback paths.
-

## 13. Verification & Validation Strategy

### 13.1 Requirements Traceability

Each requirement from the BRS/SRS will map to:

- A subsystem (Functional Architecture)
- A verification method (unit/integration/e2e/security)
- A documented test case and expected result

### 13.2 Verification Strategy

- Unit tests for core services (task state machine, policy rules)
- Integration tests (n8n callbacks, connectors)
- End-to-end tests for MVP templates
- Security testing (RBAC, tenant isolation, secrets)
- Performance tests (workflow throughput, AI latency)

### 13.3 Acceptance Criteria

- End-to-end completion for MVP templates with audit trail.
  - Approval pause/resume works reliably.
  - No cross-tenant data exposure.
  - Logged evidence for every critical action.
- 

## 14. Transition to Detailed Design

This conceptual design transitions into detailed design by specifying:

- Physical architecture and service topology
  - Database schema and indexing
  - API contracts and payload schemas
  - n8n workflow standards and reusable nodes
  - Policy catalog and approval matrices
  - Interface Control Documents (ICDs) for key integrations
- 

## 15. Appendices

- Appendix A: Context Diagram (to be produced)
- Appendix B: Functional Decomposition Diagram (to be produced)
- Appendix C: Logical Architecture Diagram (to be produced)

- Appendix D: Data Model Concept Diagram (to be produced)
  - Appendix E: Requirements Traceability Matrix Template (to be produced)
- 

END OF DOCUMENT